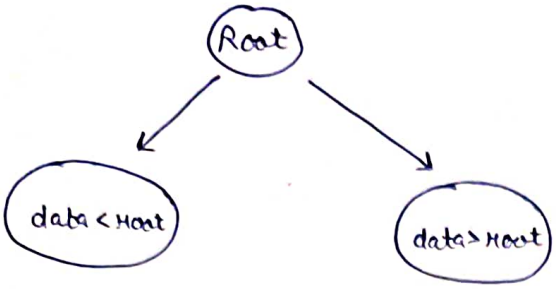
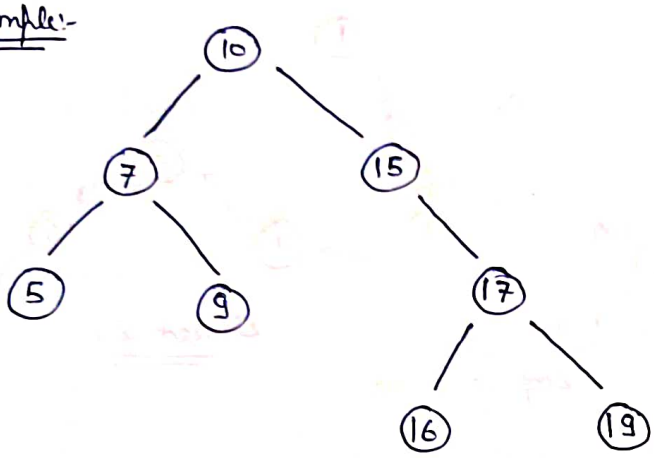
Binary Search Tree



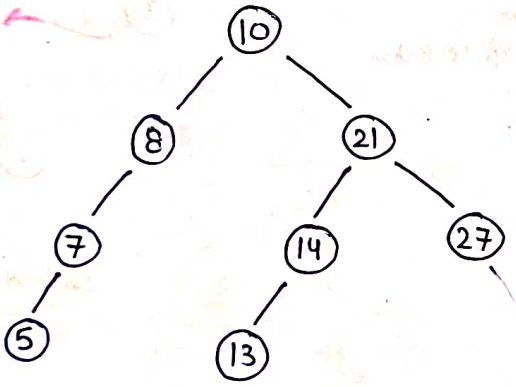
For every node:-
left subtree ka data root se chota hoga aur right subtree ka data root se jada hoga

Example:-



Insert a node in BST

10 8 21 7 27 5 14 13



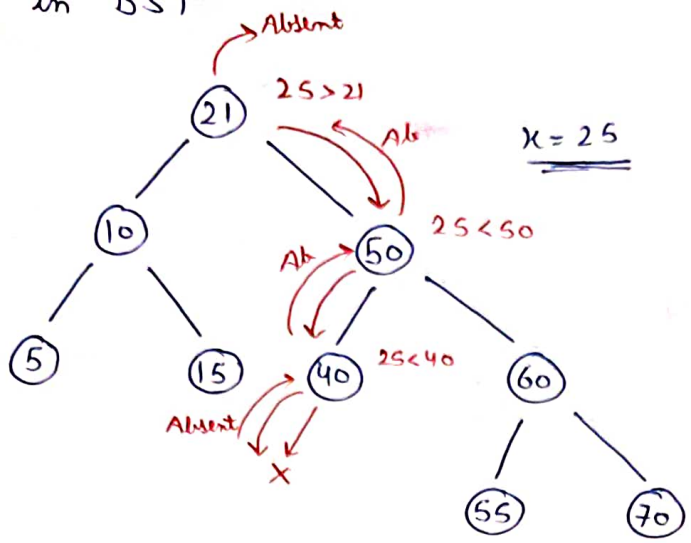
T.C. = O(log n)

```
class Node {
    public:
        int data;
        Node* left;
        Node* right;
        Node(int d) {
            this -> data = d;
```

```
        this -> left = NULL;
        this -> right = NULL;
    }
};
Void takeInput (Node* & root) {
    int data;
    cin >> data;
    while (data != -1) {
        insertIntoBST (root, data);
        cin >> data;
    }
}
Node* insertIntoBST (Node* root, int data) {
    if (root == NULL) {
        root = new Node(data);
        return root;
    }
    if (data > root -> data) {
        root -> right = insertIntoBST (root -> right, data);
    }
    else {
        root -> left = insertIntoBST (root -> left, data);
    }
    return root;
}
int main() {
    Node* root = NULL;
    takeInput (root);
    return 0;
}
```

Qu- Search in BST

i/p ⇒



x = 25

o/p ⇒ False

```
bool searchInBST(BinaryTreeNode<int> * root, int x) {
    if (root == NULL) {
        return false;
    }
    if (root -> data == x) {
        return true;
    }
    if (root -> data > x) {
        return searchInBST(root -> left, x);
    }
    else {
        return searchInBST(root -> right, x);
    }
}
```

T.C. = O(n)

S.C. = O(n)

Iterative sol^{n} :-

```
bool searchInBST(BinaryTreeNode<int> * root, int x) {
    BinaryTreeNode<int> * temp = root;
    while (temp != NULL) {
        if (temp -> data == x) {
            return true;
        }
```

```
        if (temp -> data > x) {
            temp = temp -> left;
        }
        else {
            temp = temp -> right;
        }
    }
    return false;
}
```

→ Inorder of BST is sorted

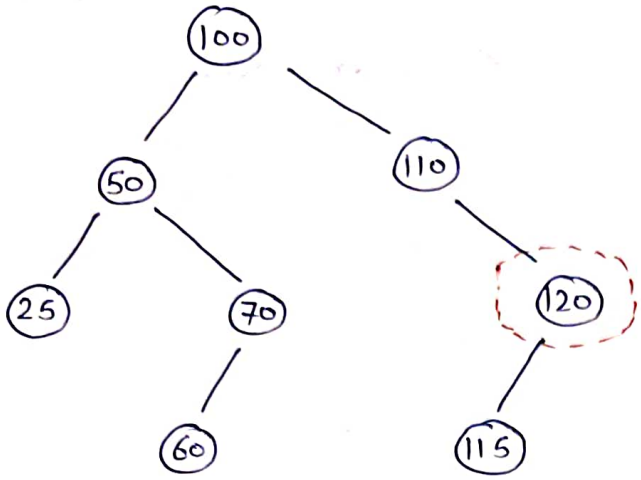
Min / Max in Binary Search Tree

```
Node* minVal (Node* root) {
    Node* temp = root;
    while (temp -> left != NULL) {
        temp = temp -> left;
    }
    return temp;
}
Node* maxVal (Node* root) {
    Node* temp = root;
    while (temp -> right != NULL) {
        temp = temp -> right;
    }
    return temp;
}
```

H.W.

Write code for finding inorder predessesor and inorder successor

Delete a Node in BST



Node to Delete = 120

Algo:-

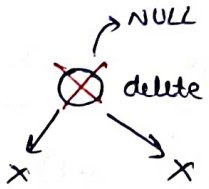
↳ Reach that node

node To Delete

↳ 0 child

↳ delete node;

return NULL



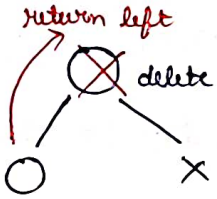
↳ 1 child

↳ left child

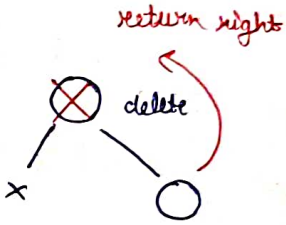
↳ temp = root->left;

delete root;

return temp;



or



↳ right child

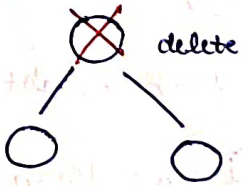
↳ temp = root->right;

delete root;

return temp;

↳ 2 child

① left me se max. Value lao aur us node ko delete kar do



Or

② right subtree me se min. Value lao aur us node ko delete kar do

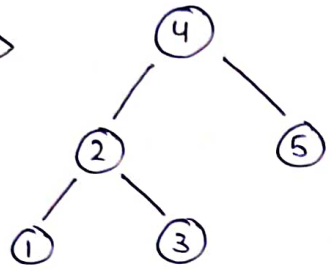
```
Node* deleteFromBST (Node* root, int val) {

    if (root == NULL) {

        return root;

    }
```

```
if (root -> data == val) {
    //0 child
    if ( root -> left == NULL && root -> right == NULL) {
        delete root;
        return NULL;
    }
    // 1 child
    if ( root -> left != NULL && root -> right == NULL) {
        Node* temp = root -> left;
        delete root;
        return temp;
    }
    if ( root -> left == NULL && root -> right != NULL) {
        Node* temp = root -> right;
        delete root;
        return temp;
    }
    // 2 child
    if ( root -> left != NULL && root -> right != NULL) {
        int mini = minVal ( root -> right) -> data;
        root -> data = mini;
        root -> right = deleteFromBST ( root -> right, mini);
        return root;
    }
}
else if (root -> data > val) {
    root -> left = deleteFromBST ( root -> left, val);
    return root;
}
```

```
else {
    root->right = deleteFromBST(root->right, val);
    return root;
}
}
```

Qu- Validate BST (Valid BST or not)

i/p =>

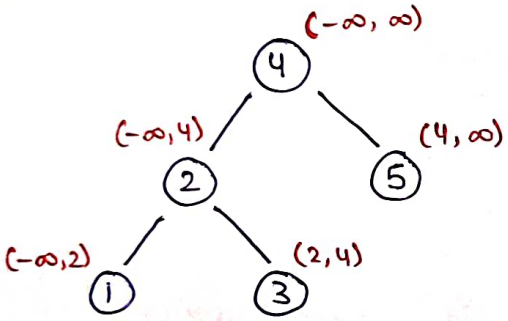


o/p => True

Approach ①

If inorder traversal is sorted → True otherwise False

Approach ②



By checking range of data

T.C. = O(n)

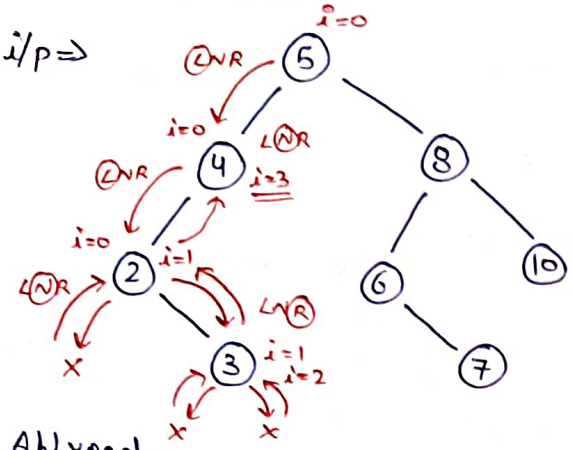
S.C. = O(H)

```
bool isBST(BinaryTreeNode<int> * root, int min, int max) {
    if (root == NULL) {
        return true;
    }
    if (root->data >= min && root->data <= max) {
        bool left = isBST(root->left, min, root->data);
        bool right = isBST(root->right, root->data, max);
        return left && right;
    }
    else {
        return false;
    }
}
```

```
bool validateBST(BinaryTreeNode<int>* root) {
    return isBST(root, INT_MIN, INT_MAX);
}
```

Qu- Kth smallest element in BST

i/p =>



K = 3

o/p => 4

When i = K return node -> data

Approach

inorder Traversal => 2 3 4 5 6 7 8 10 (ans = 4, K=3)

```
int solve(BinaryTreeNode<int>* root, int &i, int K) {
    if (root == NULL) {
        return -1;
    }
    int left = solve(root->left, i, K);
    if (left != -1) {
        return left;
    }
    i++;
    if (i == K) {
        return root->data;
    }
    return solve(root->right, i, K);
}
int KthSmallest(BinaryTreeNode<int>* root, int K) {
    int i = 0;
    int ans = solve(root, i, K);
    return ans;
}
```

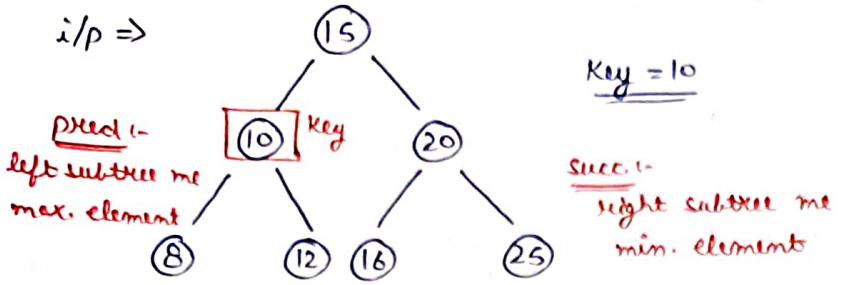
T.C. = O(n)

S.C. = O(H)

This code can be optimized by Morris Traversal

Qu- Predecessor and Successor in BST

i/p =>



Key = 10

o/p => 8 12

Approach (1)

inorder Traversal => 8 10 12 15 16 20 25

↑ ← Key → ↑
pred. succ.

T.C = O(n)
S.C. = O(n)

Approach (2)

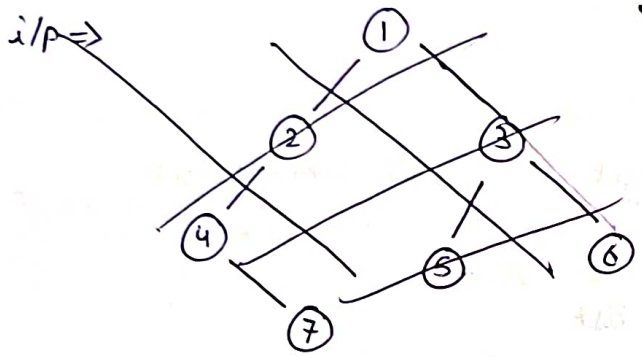
↳ node find (key)
↳ get pred. & succ. by using min/max

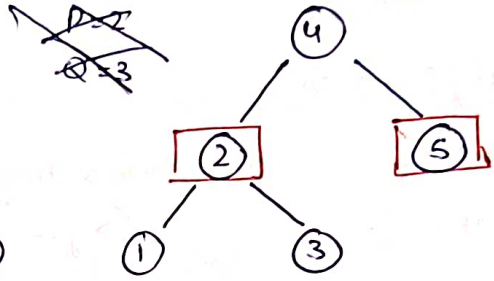
```
pair<int, int> predecessorSuccessor(TreeNode* root, int key) {
    BinaryTreeNode<int>* temp = root;
    int pred = -1;
    int succ = -1;

    while (temp->data != key) {
        if (temp->data > key) {
            succ = temp->data;
            temp = temp->left;
        }
        else {
            pred = temp->data;
            temp = temp->right;
        }
    }
    BinaryTreeNode<int>* leftTree = temp->left;
    while (leftTree != NULL) {
        pred = leftTree->data;
        leftTree = leftTree->right;
    }
```

T.C. = O(n)
S.C. = O(1)

```
BinaryTreeNode<int> * rightTree = temp -> right;
while (rightTree != NULL) {
    succ = rightTree -> data;
    rightTree = rightTree -> left;
}
pair<int, int> ans = make_pair (pred, succ);
return ans;
}
```

Qu- LCA in a BST





LCA (2,5) = 4

Approach :-

① root < a && root < b
↳ right me jana Hai

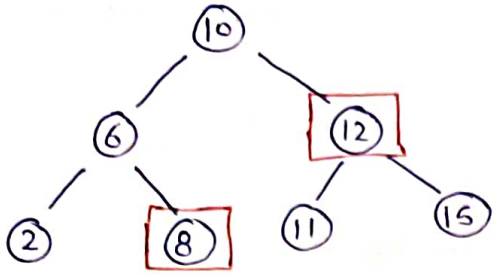
② root > a && root > b
↳ left me jana Hai

③ root > a && root < b or root < a && root > b
↳ return root

```
TreeNode<int>* LCA in a BST (TreeNode<int>* root, TreeNode<int>* P, TreeNode<int>* Q) {
    if (root == NULL) {
        return NULL;
    }
    if (root -> data < P -> data && root -> data < Q -> data) {
        return LCA in a BST (root -> right, P, Q);
    }
    if (root -> data > P -> data && root -> data > Q -> data) {
        return LCA in a BST (root -> left, P, Q);
    }
    return root;
}
```

Qu- Two Sum in a BST

i/p ⇒



Target = 20

o/p ⇒ True

Approach :-

Inorder Traversal :- 2 6 8 10 11 12 15 (i = 2, j = 15)

Two pointer approach

arr[i] + arr[j] → sum

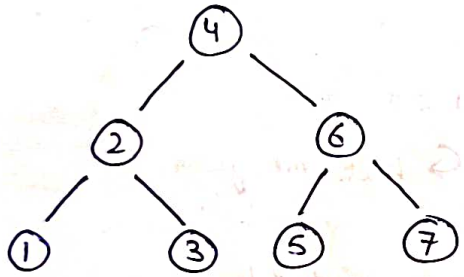
(i) sum == target → return ans

(ii) sum > target → j--

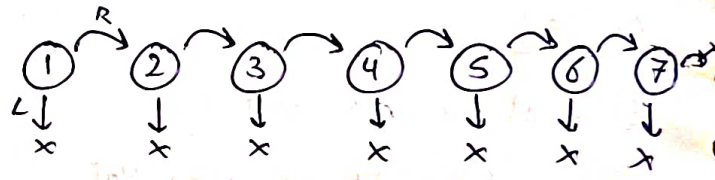
(iii) sum < target → i++

Qu- Flatten BST to a sorted linked list

i/p ⇒



o/p ⇒



Approach:-

① Store inorder Traversal (vector < Node* >) S.C. = O(n)

② left pointer null & right pointer → next element T.C. = O(n)

③ last element left & right both null

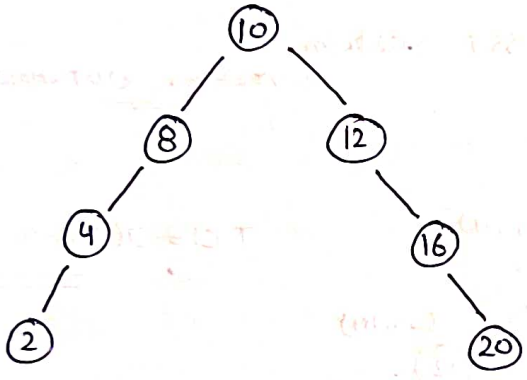
```
TreeNode<int> * flatten (TreeNode<int> * root) {
    vector<int> inorderVal;
    inorder(root, inorderVal);        } Function for inorder Traversal
    TreeNode<int>* newRoot = new TreeNode<int>(inorderVal[0]);
    TreeNode<int> * curr = newRoot;
    int n = inorderVal.size();
```

```
for (int i=1; i<n; i++) {
        TreeNode<int>* temp = new TreeNode<int>(inorderVal[i]);
        curr -> left = NULL;
        curr -> right = temp;
        curr = temp;
    }
    curr -> left = NULL;
    curr -> right = NULL;
    return newRoot;
}
```

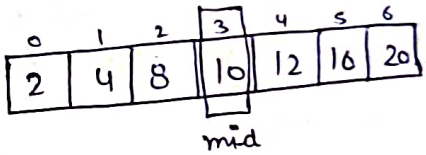
Qu- Normal BST to Balanced BST i.e. For every node abs.(h(left) - h(right)) <=

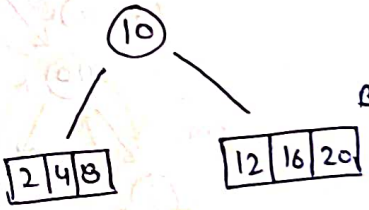
Approach:-

Store inorder and make BST from that



inorder ⇒





By Recursion

```
TreeNode<int>* inorderToBST(int s, int e, vector<int>& inorderVal) {
    if (s > e) {
        return NULL;
    }
    int mid = (s+e)/2;
    TreeNode<int>* root = new TreeNode<int>(inorderVal[mid]);
    root -> left = inorderToBST(s, mid-1, inorderVal);
    root -> right = inorderToBST(mid+1, e, inorderVal);
    return root;
}
```

```
TreeNode<int>* balancedBST(TreeNode<int>* root) {
    vector<int> in;
    inorder(root, in);
    return inorderTOBST(0, in.size()-1, in);
}
```

Qu- BST from Preorder Traversal

Approach ①:-

preorder is given & get inorder by sorting preorder then make BST as previously done in Binary Trees T.C. = O(n log n)

Approach ②:-

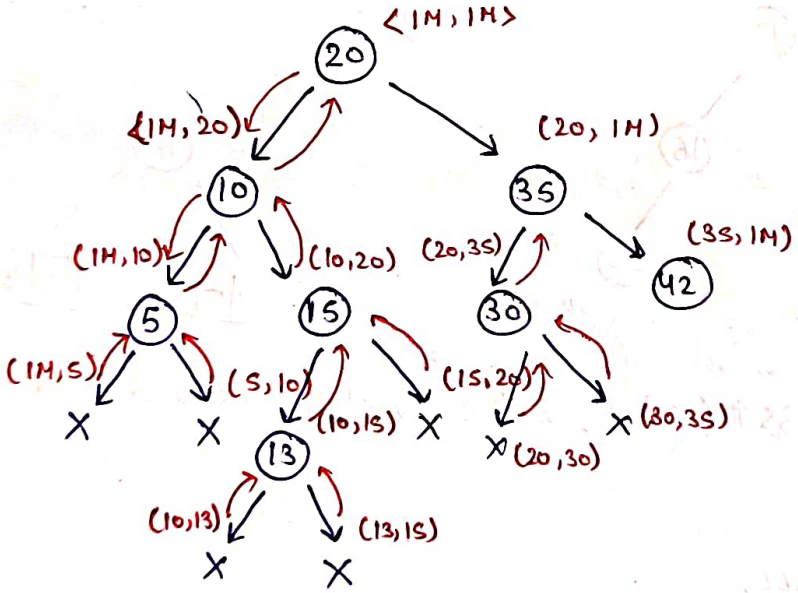
Preorder (NLR) :-

0	1	2	3	4	5	6	7
20	10	5	15	13	35	30	42

Traverse

Solving using range like in valid BST question

<INT-Min> <INT-Max>



T.C. = O(3N) ≈ O(n)

```
BinaryTreeNode<int>* solve(vector<int>& preorder, int mini, int maxi, int& i) {
    if (i >= preorder.size()) {
        return NULL;
    }
    if (preorder[i] < mini || preorder[i] > maxi) {
        return NULL;
    }
```

```
    BinaryTreeNode<int>* root = new BinaryTreeNode<int>(preorder[i++]);
    root->left = solve(preorder, mini, root->data, i);
    root->right = solve(preorder, root->data, maxi, i);
    return root;
}
BinaryTreeNode<int>* preorderToBST(vector<int>& preorder) {
    int mini = INT_MIN;
    int maxi = INT_MAX;
    int i = 0;
    return solve(preorder, mini, maxi, i);
}
```

Qu- Merge 2 BST

Approach (1) :-

BST 1 , BST 2

inorder 1 inorder 2

T.C. $\Rightarrow O(n+m)$

S.C. $= O(n+m)$

Merge both inorder in sorted manner

Now make BST from inorder traversal as previously done

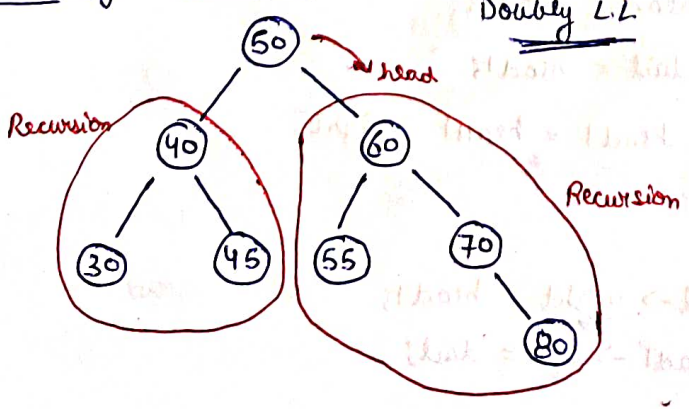
Approach (2) :-

(1) BST 1 $\longrightarrow$ Sorted Linked List
BST 2 $\longrightarrow$ Sorted Linked List
} Convert

(2) Merge 2 sorted L.L.

(3) Sorted L.L. $\longrightarrow$ BST

Converting BST to LL



Doubly L.L

Algo:-

(i) Convert right subtree to L.L.

(ii) root->right = head
head->left = root

(iii) head = root

(iv) Convert left subtree to L.L.

```
void convertIntoSortedDLL(TreeNode<int>* root, TreeNode<int>*& head) {
    if (root == NULL) {
        return;
    }
    convertIntoSortedDLL(root->right, head);
    root->right = head;
    if (head != NULL) {
        head->left = root;
    }
    head = root;
    convertIntoSortedDLL(root->left, head);
}
```

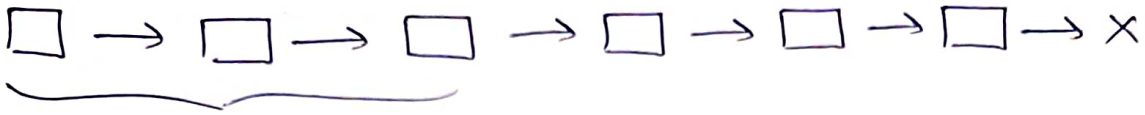
Merging 2 LL

inplace merge

```
TreeNode<int> mergeLinkedList(TreeNode<int>* head1, TreeNode<int>* head2) {
    TreeNode<int>* head = NULL;
    TreeNode<int>* tail = NULL;
    while (head1 != NULL && head2 != NULL) {
        if (head1->data < head2->data) {
            if (head == NULL) {
                head = head1;
                tail = head1;
                head1 = head1->right;
            }
            else {
                tail->right = head1;
                head1->left = tail;
```

```
                tail = head1;
                head1 = head1 -> right;
            }
        }
        else {
            if (head == NULL) {
                head = head2;
                tail = head2;
                head2 = head2 -> right;
            }
            else {
                tail -> right = head2;
                head2 -> left = tail;
                tail = head2;
                head2 = head2 -> right;
            }
        }
    }
    while (head1 != NULL) {
        tail -> right = head1;
        head1 -> left = tail;
        tail = head1;
        head1 = head1 -> right;
    }
    while (head2 != NULL) {
        tail -> right = head2;
        head2 -> left = tail;
        tail = head2;
        head2 = head2 -> right;
    }
    return head;
}
```

Converting Sorted L.L. to BST



(i) $n/2$ left tree

(ii) create root

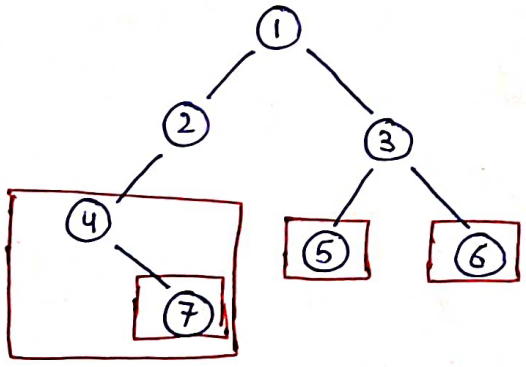
(iii) root -> left = left tree

(iv) head -> right tree

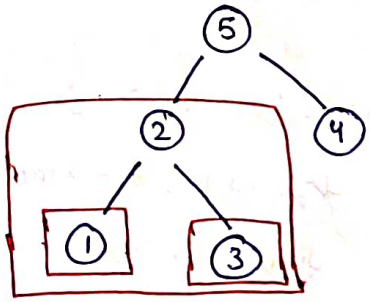
```
TreeNode<int>* sortedLLToBST(TreeNode<int>* &head, int n) {
    if (n<=0 || root == NULL) {
        return NULL;
    }
    TreeNode<int>* left = sortedLLToBST(head, n/2);
    TreeNode<int>* root = head;
    root->left = left
    head = head->next;
    root->right = sortedLLToBST(head, n-n/2-1);
    return root;
}
```

Qu- Largest BST in a Binary Tree

i/p ⇒



o/p ⇒ 2



o/p ⇒ 3

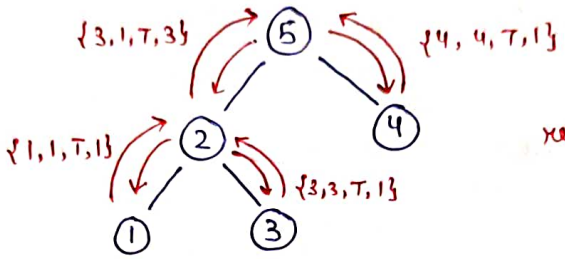
Approach ①

↳ Har node par jaunga aur check karunga ki valid BST hai ya nhi aur uska size store kar lunga. T.C. = $O(n^2)$

Approach ②

node is Valid Subtree
- → left subtree valid BST
- → right subtree valid BST
- → max. left < root -> data < right min.

info.
- → maxi
- → mini
- → is BST
- → size



return max. size

```
class info {
    public:
        int maxi;
        int mini;
        bool isBST;
        int size;
};
info solve(TreeNode<int> * root, int & ans) {
    if (root == NULL) {
        return {INT_MIN, INT_MAX, true, 0};
    }
    info left = solve(root->left, ans);
    info right = solve(root->right, ans);
    info currNode;
    currNode.size = left.size + right.size + 1;
    currNode.maxi = max(root->data, right.maxi);
    currNode.mini = min(root->data, left.mini);
```

```
if (left.isBST && right.isBST && root->data > left.maxi && root->data <
                                                    right.mini) {
        currNode.isBST = true;
    }
    else {
        currNode.isBST = false;
    }
    if (currNode.isBST) {
        ans = max(ans, currNode.size);
    }
    return currNode;
}

int largestBST(TreeNode<int>* root){
    int maxSize = 0;
    info temp = solve(root, maxSize);
    return maxSize;
}
```